

1 Comparison Between AFL++ and KLEE

1.1 Coverage achieved

Both AFL++ and KLEE achieved high coverage on the four target functions: `treetable_add()`, `treetable_get()`, `treetable_get_first_key()`, and `treetable_get_greater_than()`.

AFL++ exercised most dynamic behaviors of the implementation, including: left/right insertions, duplicate-key replacements, tree rotations, and both successful and unsuccessful lookups. The fuzzing campaign also heavily exercised the balancing and ordering oracles through long randomized execution sequences.

KLEE, however, explored edge cases more systematically. Symbolic execution was particularly effective at reaching:

- empty-table states,
- NULL-related paths,
- successor boundary conditions,
- duplicate-key corner cases.

Some branches were reached by only one technique. AFL++ explored deeper state interactions due to long random traces, while KLEE reached difficult corner cases through symbolic path exploration.

1.2 Bugs detected

Both techniques discovered the same two major bugs.

First, `treetable_get_first_key()` crashes on empty tables because `tree_min()` dereferences `sentinel->left`, which is NULL. This produced segmentation faults detected by both AFL++ and KLEE.

Second, `treetable_get_greater_than()` sometimes returned a key that was not strictly greater than the search key, violating the API contract. This issue appeared especially with duplicate keys and maximum-element queries.

AFL++ additionally exposed deeper state-related assertion failures and AddressSanitizer crashes after long operation sequences. KLEE, on the other hand, generated smaller and more precise failing inputs that were easier to analyze.

1.3 Test-generation cost

The cost of test generation differed significantly. AFL++ required:

- designing a persistent fuzzing harness,
- defining an input format,
- preparing seed inputs,
- running the fuzzer for approximately 30 minutes.

KLEE required:

- LLVM bitcode generation,
- symbolic drivers,
- symbolic variable management.

AFL++ produced a larger test suite but achieved very high execution throughput. KLEE produced fewer but more targeted tests.

1.4 Effort Required

The AFL++ harness required more engineering effort overall. The harness needed:

- a stable binary input format,
- memory management,
- invariant oracles,
- persistent-mode support,
- crash detection,
- assertions validating semantic correctness.

Designing effective fuzz inputs and useful invariants was non-trivial.

KLEE required less infrastructure code, but symbolic execution introduced its own complexity:

- symbolic variable management,
- path explosion control,
- specialized symbolic drivers for each function.

1.5 Conclusion

Overall, AFL++ was more effective for broad exploration and stress testing, while KLEE was stronger at systematically exploring semantic edge cases. In this case study, AFL++ would likely be the preferred first technique because it quickly exposed crashes and achieved excellent practical coverage.